

Le développement mobile

Table des matières

I. Contexte	3
II. Les applications natives	3
III. Exercice : Appliquez la notion	4
IV. Les applications web embarquées	5
V. Exercice : Appliquez la notion	6
VI. Les applications hybrides	7
VII. Exercice : Appliquez la notion	8
VIII. Essentiel	8
IX. Auto-évaluation	8
A. Exercice final	8
Solutions des exercices	10

I. Contexte

Durée : 1 h

Environnement de travail : Simplement un carnet de notes

Pré-requis : Les bases de JavaScript et quelques connaissances en React Web

Contexte

Le développement d'applications mobiles a connu de grands changements ces dernières années. À l'origine, deux concurrents se sont imposés sur le marché : iOS et Android. Chacun permet aux développeurs de créer leurs propres packages d'application qui peuvent être installés sur leurs terminaux respectifs.

Ces concurrents proposent de développer des applications dans des langages différents. iOS propose Objective-C et Swift, alors qu'Android propose Java et Kotlin.

Lorsque nous développons des applications initialement, il fallait donc deux projets et potentiellement deux équipes de développement pour réunir toutes les compétences requises.

Par la suite, de nouveaux outils permettant de mutualiser une partie des codes afin de rendre l'expérience de développement plus agréable et efficace sont apparus. On parle de solution multi-plateformes (*cross-platform*) : elles sont basées sur différents paradigmes.

Le développement mobile est donc aujourd'hui, grâce à ces technologies, beaucoup plus abordable et efficient.

II. Les applications natives

Objectifs

- Comprendre le fonctionnement d'une application native
- Comprendre les avantages liés aux applications natives
- Utiliser des valeurs dynamiques calculées avec un petit algorithme qui réagit aux événements, comme un clic

Mise en situation

Les applications natives pour iOS ou Android sont des packages applicatifs développés respectivement en Objective-C ou Swift pour iOS et Java ou Kotlin pour Android.

Ces langages nécessitent que le code source soit compilé afin de pouvoir être interprété par le téléphone.

Ces applications bénéficient par conséquent de tous les avantages d'un code source compilé, à savoir du typage statique fort, vérifié à la compilation, ainsi que des performances accrues pour les mêmes raisons.

Complément

Il faut savoir que ces applications natives sont la façon historique de créer des applications : il y a donc énormément de bibliothèques de code pour développer, une grande communauté, et surtout un excellent recul, car cela fait plus de 10 ans que l'on utilise ce type d'applications.

De manière générale, si l'on a un budget, une *deadline* et une équipe illimitée, l'application finale sera toujours plus performante en code natif. Mais pour produire une application sur les deux plateformes, il sera nécessaire d'investir plus de temps avec des profils de développeurs plus spécialisés pour arriver au résultat.

Le problème est donc qu'on a besoin de plusieurs équipes pour assurer le développement et qu'on peut difficilement faire basculer des ressources d'une équipe à l'autre, n'étant pas dans le même langage de programmation.

De plus, pour observer chaque modification effectuée, il faut re-compiler l'application, ce qui fait perdre beaucoup de temps aux développeurs.

Syntaxe À retenir

- Les applications natives sont la base de la programmation mobile, elles sont très souvent plus performantes du fait qu'elles utilisent du code compilé.
- Nous verrons que c'est également le socle de base de toutes les technologies « *cross-platform* ». Les applications basées sur un moteur web et/ou hybrides sont au final des applications natives qui ont un fonctionnement bien précis : rendre une *webview* ou des éléments natifs (seul le pilotage de ce rendu change).
- De plus, on peut être amené à utiliser du code natif sur les deux types d'applications que nous allons voir pour répondre à certaines problématiques, telles que l'exploitation d'API, comme la caméra du téléphone.
- Enfin, toutes les applications nécessitent un procédé de *build* (construction) où le code est compilé pour de meilleures performances.

Exercice : Appliquez la notion

[solution n°1 p.11]

Exercice

Dans quel langage les applications Android sont-elles programmées ?

- ☐ C#
- ☐ Java / Kotlin
- ☐ PHP
- ☐ JavaScript

Exercice

Globalement, vous diriez que les applications natives sont...

- ☐ Moins performantes que les applications *cross-platform*
- ☐ La seule manière de faire des applications mobiles
- ☐ Plus performantes que les applications *cross-platform*
- ☐ Possèdent un meilleur rapport qualité/prix

Exercice

Les applications mobiles utilisent...

- ☐ Du code compilé
- ☐ Du code non-compilé

IV. Les applications web embarquées

Objectifs

- Comprendre le principe d'une application web embarquée
- Comprendre les avantages et les inconvénients de ce mode de développement
- Comprendre le contexte autour de la création de cette technologie

Mise en situation

Comme nous l'avons vu, la solution historique réside dans le développement d'applications natives. Cependant, ces applications présentent plusieurs difficultés, notamment sur l'expérience de développement, malgré des performances optimales.

De là sont nées les applications basées sur un navigateur web, telles que Phonegap ou, plus tard, Apache Cordova, afin d'apporter une première réponse aux problématiques rencontrées par les applications natives.

Le principe de ces outils est de programmer une application dans les technologies du Web, à l'aide de JavaScript et en utilisant (ou pas) un *framework* du type de ReactJS, Angular ou VueJS, par exemple.

Cette application est ensuite créée de façon à fonctionner hors-ligne, puis on embarque son code dans une application native « simple et classique » qui ne fait qu'afficher une *webview* grâce à du code natif géré par le framework (comprendre « une page web dans l'application »).

Remarque

C'est en quelque sorte l'homonyme d'une *iframe* dans le monde du développement web.

Pour accéder aux API système natives du téléphone, telles que la caméra ou le GPS, les frameworks Cordova ou Phonegap mettent à disposition une API JavaScript qui peut être utilisée dans le navigateur web via le JavaScript.

Sous le capot, ils appellent du code natif Java ou Objective-C et font communiquer ce code avec le moteur JavaScript de la *webview*. On a donc un site web embarqué affiché dans une application mobile.

Cela présente plusieurs avantages :

- D'une part, la mise en page se fait grâce à du HTML et du CSS, qui sont réputés pour être de bons outils pour créer des interfaces graphiques.
- De plus, on peut capitaliser sur les compétences d'une seule équipe de développeurs web qui pourra très facilement faire du portage sur mobile. On pourrait même n'avoir qu'une seule et unique application pour le Web et pour mobile si c'est bien réalisé.
- Enfin, on profite de tous les outils liés à l'écosystème JavaScript, tels que le rechargement à chaud, et on peut donc constater en temps réel les modifications effectuées en développement, contrairement à une application native où il faut recompiler à chaque fois.

Sur le papier, c'est donc génial ! Cependant, après plusieurs observations, des géants tels que Facebook ont abandonné l'idée, se rendant compte que les performances ne sont pas au rendez-vous du fait que l'application tourne dans une page web mobile dans le navigateur du système d'exploitation.

En effet, on constate assez facilement que l'on se trouve dans une application qui affiche une *webview*, et elle ne semblera généralement pas très fluide.

La compatibilité est aujourd'hui sur le déclin du fait des nouvelles technologies, et, globalement, Apple n'aime pas trop ce type d'application. Il faut donc bien la peaufiner afin de la rendre propre, sous peine de ne pas la voir se faire publier sur les stores pour iPhone.

Cependant, pour créer une application simplement, et si l'on n'a pas les contraintes d'un géant du Web, cela peut être un choix intelligent. Notamment pour déployer une application mobile en même temps que son homologue web.

Une référence dans ce monde est le framework **ionic**¹.

Syntaxe À retenir

- Les applications web embarquées telles que celles générées par le framework Cordova sont une première réponse aux problématiques posées depuis des années par le développement natif.
- Elles améliorent l'expérience de développement et permettent de mutualiser du code et des compétences d'équipe, ce qui améliore la productivité globale d'un projet !
- Cependant, elles souffrent de certains problèmes, notamment de performances. C'est pourquoi certains géants qui s'y sont risqués ont décidé de basculer sur du natif uniquement, ou sur du *cross-platform*.

Complément Liens utiles

- <https://ionicframework.com/>
- <https://cordova.apache.org/>

Exercice : Appliquez la notion

[solution n°2 p.11]

Exercice

À laquelle de ces problématiques ne répond pas une application web embarquée ?

- ☐ Mutualiser le code entre les projets web et mobile
- ☐ Avoir de bonnes performances
- ☐ Des outils simplifiés pour la mise en page
- ☐ Réduire le nombre de compétences nécessaires dans une équipe

Exercice

Lequel de ces frameworks ne permet pas de réaliser des applications web embarquées ?

- ☐ Ionic
- ☐ Cordova
- ☐ PhoneGap
- ☐ Symfony

Exercice

Quelle technologie est utilisée pour la mise en page d'applications web embarquées ?

- ☐ HTML / CSS
- ☐ Objective-C
- ☐ Kotlin

¹ <https://ionicframework.com/>

VI. Les applications hybrides

Objectif

- Comprendre l'intérêt et le principe de base d'une application hybride

Mise en situation

Nous avons vu les applications web embarquées et les applications natives. Nous avons désormais un troisième type d'applications cherchant à solutionner et obtenir le meilleur des deux mondes (web et mobile) : ce sont les applications hybrides.

Pour pallier le souci de performances, des frameworks tels que React Native, Flutter ou encore NativeScript partent du principe que l'on peut écrire du code proche d'une syntaxe qui se fait habituellement dans le Web (déclarative), qui servira à piloter un rendu en code natif.

Qu'est-ce que ça veut dire ?

Concrètement, on prend le meilleur du Web et on se débrouille pour en faire du code qui vient générer à la volée du code natif, ou du moins des instructions pour un rendu natif *cross-platform*.

Là où chaque framework a sa technique, Flutter va par exemple reproduire les composants natifs via un moteur 3D, comme le ferait un jeu vidéo ; React Native, lui, va envoyer des instructions du moteur JavaScript vers le code natif, qui va se charger d'afficher des composants natifs iOS ou Android en fonction de la plateforme.

On profite donc de tous les avantages du Web et du natif, ou du moins on obtient un bon compromis. Tout comme Cordova, on peut accéder aux différentes API système, telles que la caméra ou le GPS, généralement via des packages *open-source* ou intégrés à React Native directement.

Si jamais il venait à manquer un de ces packages, on pourrait le développer soi-même, mais il faudra cependant coder du Java ou de l'Objective-C. C'est donc assez extensible, si besoin.

Ce cours permet d'apprendre à utiliser React Native pour développer des applications mobiles *cross-platform* performantes.

Un peu plus sur React Native

Idée initialement développée lors d'un hackaton, React Native a su prendre de l'ampleur et est désormais soutenu dans son développement et utilisé par Facebook et Instagram.

React Native s'inspire de son homologue web et fournit des composants spécifiques pour afficher du code natif au lieu d'HTML. Par exemple, on utilisera un composant `View` plutôt qu'une `div` pour définir un conteneur.

React Native est une véritable révolution, car c'est le premier « vrai » framework à proposer une expérience de développement riche avec des performances vraiment intéressantes.

De plus, l'écosystème React étant très fourni, on peut s'appuyer sur une quantité de ressources assez conséquente pour utiliser React Native et également obtenir de l'aide.

Cela en fait un outil de choix de par son écosystème et son image dans le monde du développement mobile. Il est d'ailleurs utilisé en production par de nombreuses entreprises très connues de la « Tech », telles que Uber, AirBnB ou encore Tesla.

Syntaxe **À retenir**

- Les applications hybrides sont aujourd'hui la dernière évolution du développement *cross-platform*. Elles permettent d'atteindre un bon compromis entre performances et facilité de développement.
- C'est une technologie basée sur l'utilisation d'un langage proche de celui du Web, grâce à sa forme permettant de générer du code natif performant.

Exercice : Appliquez la notion

[solution n°3 p.12]

Exercice

Quel est l'équivalent d'une `div` en React Native ?

- ☐ Box
- ☐ View
- ☐ Container

Exercice

En termes de date d'apparition, vous diriez que React Native, par rapport aux applications natives et web embarquées, est...

- ☐ La plus ancienne façon de créer des applications mobiles
- ☐ La plus récente façon de créer des applications mobiles
- ☐ Ni l'une, ni l'autre

Exercice

Laquelle de ces phrases est fausse ?

- ☐ React Native utilise un *thread* JavaScript pour faire tourner son code
- ☐ React Native utilise un moteur de rendu 3D avec un *canva* pour afficher ses composants
- ☐ NativeScript est un framework pour créer des applications *cross-plaform* hybrides

VIII. Essentiel

IX. Auto-évaluation

A. Exercice final

Exercice 1

[solution n°4 p.13]

Exercice

Quelle est la manière historique de créer des applications mobiles ?

- ☐ Les applications hybrides
- ☐ Les applications natives
- ☐ Les application web embarquées
- ☐ Les progressives web apps (PWA)

Exercice

Quel est le principal défaut des applications web embarquées ?

- ☐ La difficulté à programmer la mise en page
- ☐ Le temps de compilation de l'application
- ☐ Les performances amoindries du fait que cela tourne dans une *webview*
- ☐ Le temps de montée en compétences sur la technologie Cordova

Exercice

Le budget et la *deadline* ne sont pas un problème pour mon entreprise ; de plus, nous n'avons pas une application très compliquée. Par contre, nous recherchons des performances au top. Quel est le type d'application le plus adapté à mon besoin ?

- ☐ Les applications hybrides
- ☐ Les applications natives
- ☐ Les application web embarquées
- ☐ Les progressives web apps (PWA)

Exercice

Il est possible d'accéder aux API natives (caméra, GPS, etc.) depuis une application web embarquée.

- ☐ Vrai
- ☐ Faux

Exercice

Quelle est la société qui appuie principalement le développement de React Native ?

- ☐ PhoneGap
- ☐ Tesla
- ☐ Facebook
- ☐ Apple

Exercice

Pourquoi utiliser un framework hybride comme React Native plutôt qu'une application native ou qu'une application web embarquée ?

- ☐ C'est plus à la mode
- ☐ Pour réunir le meilleur des deux mondes (Web et mobile)
- ☐ Je souhaiterais ne pas avoir à configurer le système de *build* de l'application

Exercice

Laquelle de ces technologies ne permet pas de réaliser des applications hybrides ?

- ☐ Flutter
- ☐ NativeScript
- ☐ Cordova
- ☐ React Native

Exercice

Il est possible d'étendre React Native.

- ☐ Vrai
- ☐ Faux

Exercice

Laquelle de ces paires n'est pas du même type d'application ?

- ☐ Flutter / NativeScript
- ☐ Cordova / PhoneGap
- ☐ Native Android / Native iOS
- ☐ Cordova / React Native

Solutions des exercices

Exercice p. 4 Solution n°1**Exercice**

Dans quel langage les applications Android sont-elles programmées ?

- ☐ C#
- ☒ Java / Kotlin
- ☐ PHP
- ☐ JavaScript
-  Les applications Android sont programmées soit en Java, soit en Kotlin, qui va lui-même être compilé vers du code Java.

Exercice

Globalement, vous diriez que les applications natives sont...

- ☐ Moins performantes que les applications *cross-platform*
- ☐ La seule manière de faire des applications mobiles
- ☒ Plus performantes que les applications *cross-platform*
- ☐ Possèdent un meilleur rapport qualité/prix
-  Les applications mobiles sont la couche la plus basse de la programmation mobile, elles sont donc généralement plus performantes de par leur code compilé. Cependant, il faut des développeurs multi-compétences ou plusieurs équipes, si l'on souhaite programmer sur plusieurs plateformes.

Exercice

Les applications mobiles utilisent...

- ☒ Du code compilé
- ☐ Du code non-compilé
-  Elles utilisent du code compilé, et c'est ce qui leur permet d'avoir de meilleures performances grâce aux optimisations de compilation.

Exercice p. 6 Solution n°2**Exercice**

À laquelle de ces problématiques ne répond pas une application web embarquée ?

- ☐ Mutualiser le code entre les projets web et mobile
- ☒ Avoir de bonnes performances
- ☐ Des outils simplifiés pour la mise en page
- ☐ Réduire le nombre de compétences nécessaires dans une équipe

Q Les applications web embarquées solutionnent la mutualisation du code en utilisant des frameworks web tels que ReactJS. Mais également la mise en page, grâce à HTML et CSS. De manière globale, on réduit donc le nombre de compétences nécessaires, car on peut utiliser les compétences web pour faire du mobile. Cela au prix d'une UX et de performances sensiblement dégradées.

Exercice

Lequel de ces frameworks ne permet pas de réaliser des applications web embarquées ?

- ☐ Ionic
- ☐ Cordova
- ☐ PhoneGap
- ☒ Symfony

Q Ionic est une surcouche à Cordova, qui est lui-même une amélioration de PhoneGap. Ils permettent donc tous de créer des applications web embarquées, là où Symfony est un framework d'application web PHP qui n'a rien à voir avec ce monde.

Exercice

Quelle technologie est utilisée pour la mise en page d'applications web embarquées ?

- ☒ HTML / CSS
- ☐ Objective-C
- ☐ Kotlin

Q Les applications web embarquées utilisent des technologies du Web, donc HTML / CSS.

Exercice p. 8 Solution n°3

Exercice

Quel est l'équivalent d'une `div` en React Native ?

- ☐ Box
- ☒ View
- ☐ Container

Q Une `View` est l'homologue d'une `div` en React Native et permet de définir un conteneur.

Exercice

En termes de date d'apparition, vous diriez que React Native, par rapport aux applications natives et web embarquées, est...

- ☐ La plus ancienne façon de créer des applications mobiles
- ☒ La plus récente façon de créer des applications mobiles
- ☐ Ni l'une, ni l'autre

Q React Native et les applications hybrides sont la façon la plus récente de créer des applications mobiles *cross-platform* en joignant le meilleur du monde natif et du Web.

Exercice

Laquelle de ces phrases est fausse ?

- ☒ React Native utilise un *thread* JavaScript pour faire tourner son code
- ☐ React Native utilise un moteur de rendu 3D avec un *canva* pour afficher ses composants
- ☐ NativeScript est un framework pour créer des applications *cross-plaform* hybrides

Exercice p. 8 Solution n°4**Exercice**

Quelle est la manière historique de créer des applications mobiles ?

- ☐ Les applications hybrides
 - ☒ Les applications natives
 - ☐ Les application web embarquées
 - ☐ Les progressives web apps (PWA)
-  Historiquement, ce sont les applications natives qui étaient la seule façon de faire. D'ailleurs, les deux autres que nous avons vues utilisent leur socle de base pour s'afficher.

Exercice

Quel est le principal défaut des applications web embarquées ?

- ☐ La difficulté à programmer la mise en page
 - ☐ Le temps de compilation de l'application
 - ☒ Les performances amoindries du fait que cela tourne dans une *webview*
 - ☐ Le temps de montée en compétences sur la technologie Cordova
-  Les performances sont amoindries, car l'application entière est un site web embarqué qui tourne hors-ligne dans un navigateur web, ce qui ne permet pas aux composants du téléphone de faire tourner du code ultra-optimisé par un procédé de compilation.

Exercice

Le budget et la *deadline* ne sont pas un problème pour mon entreprise ; de plus, nous n'avons pas une application très compliquée. Par contre, nous recherchons des performances au top. Quel est le type d'application le plus adapté à mon besoin ?

- ☐ Les applications hybrides
 - ☒ Les applications natives
 - ☐ Les application web embarquées
 - ☐ Les progressives web apps (PWA)
-  Tant que la *codebase* n'est pas trop grosse et que l'application ne met donc pas trop de temps à compiler à chaque modification, ce qui impacterait l'expérience de développement, alors, avec ces conditions, les applications natives sont plus adaptées.

Sachez également que l'on peut mixer une application native avec certaines parties en React Native, par exemple.

Exercice

Il est possible d'accéder aux API natives (caméra, GPS, etc.) depuis une application web embarquée.

☒ Vrai

☐ Faux

 L'application web embarquée peut, par le biais de l'API de Cordova ou Phonegap par exemple, accéder à du code natif et donc aux composants comme la caméra et le GPS depuis le JavaScript du navigateur.

Exercice

Quelle est la société qui appuie principalement le développement de React Native ?

☐ PhoneGap

☐ Tesla

☒ Facebook

☐ Apple

 C'est Facebook, suite à la réalisation d'un prototype lors d'un hackathon par des étudiants.

Exercice

Pourquoi utiliser un framework hybride comme React Native plutôt qu'une application native ou qu'une application web embarquée ?

☐ C'est plus à la mode

☒ Pour réunir le meilleur des deux mondes (Web et mobile)

☐ Je souhaiterais ne pas avoir à configurer le système de *build* de l'application

 Malheureusement, le *build* est une étape incontournable dans le développement hybride également, car il se base sur une application native en socle. Vous pourriez avoir des actions chez Facebook, mais c'est surtout car, grâce à cela, on réunit le meilleur des deux mondes, avec une expérience de développement aux petits oignons et des performances tout à fait correctes. Le tout sans limiter les prétentions futures de l'application.

Exercice

Laquelle de ces technologies ne permet pas de réaliser des applications hybrides ?

☐ Flutter

☐ NativeScript

☒ Cordova

☐ React Native

 Cordova permet de réaliser des applications web embarquées.

Exercice

Il est possible d'étendre React Native.

- ☒ Vrai
- ☐ Faux
-  Il est possible d'étendre React Native en utilisant ou en programmant des packages natifs.

Exercice

Laquelle de ces paires n'est pas du même type d'application ?

- ☐ Flutter / NativeScript
- ☐ Cordova / PhoneGap
- ☐ Native Android / Native iOS
- ☒ Cordova / React Native
-  Cordova permet de réaliser des applications web embarquées, là où React Native permet de réaliser des applications hybrides.